SPRINT 3 REVIEW DOCUMENT

Project: TripRoute (Travel Assistant)

Team 5 Members:

- Sameer Maayiz
- Dongwan Kim
- Mainuddin

Overview

Sprint 3 transformed TripRoute into a production-ready AI-powered travel planning system by migrating from ChatGPT to Gemini AI, building a FastAPI production server with session management, developing a complete frontend interface with interactive maps and route visualization, and enhancing route optimization across four transportation modes. The sprint achieved seamless integration between Gemini's Google Search grounding, structured data extraction, and Google Maps APIs to provide real-time, accurate trip recommendations with optimized multi-modal routing. The result is a fast, reliable, and intuitive end-to-end trip planning experience.

What was completed in this sprint?

Frontend

- Developed complete HTML/CSS/JavaScript interface (demo.html) with responsive 3-column layout
- Implemented chat interface with streaming message support and loading indicators
- Integrated Google Maps JavaScript API for interactive map display
- Created dynamic place markers with info windows showing place details
- Built route polyline rendering with color-coded paths for each transportation mode
- Developed places list view with collapsable route details for each leg
- Implemented transport mode switcher (driving, walking, bicycling, transit)
- Created draggable column splitters for customizable layout
- Added print functionality for trip itineraries
- Designed responsive layout adapting to desktop and mobile screens

Backend & API

- Built production FastAPI server (server.py) with comprehensive endpoints
- Implemented in-memory session storage with unique session IDs for multi-user support
- Created /chat endpoint with streaming responses using Gemini 2.5 Flash Lite
- Developed /optimize endpoint for route generation across 4 transport modes
- Added /session endpoint for session initialization with welcome messages
- Configured CORS middleware for cross-origin frontend-backend communication
- Implemented background task for asynchronous structured data extraction
- Integrated Gemini Google Search grounding for real-time place information
- Added comprehensive error handling and logging throughout API

AI Integration & Structured Extraction

- Migrated from OpenAI ChatGPT to Google Gemini 2.5 Flash Lite
- Achieved 95% response time reduction (from 10-30s to 1-2s)
- Implemented Google Search grounding with enforced prompts for every place recommendation
- Created Pydantic models for structured output (TripRouteRecommendations, TripMeta, RecommendationItem)
- Developed Place ID extraction from grounding metadata for reliable Google Maps integration
- Built conversation-to-JSON extraction system running in background while user chats
- Implemented streaming for both chat responses and structured extraction

Route Optimization

- Enhanced optimization.py to support all 4 transportation modes (driving, walking, bicycling, transit)
- Implemented Place ID normalization for reliable matching between Gemini and Google Maps
- Developed special transit handling (two-step: driving optimization for waypoint order, then leg-by-leg transit)
- Integrated Google Maps Place Details API for place enrichment
- Created structured direction output with leg-by-leg steps including distance and duration
- Implemented step normalization for nested transit steps (bus, subway, train details)
- Generated 4 separate JSON output files per optimization (one per transport mode)
- Added departure time support for future transit schedules

Documentation

- Created SETUP_GUIDE.md with comprehensive environment configuration
- Wrote RUNNING_LOCALLY.md for development workflow

- Documented API endpoints and response formats in server.py docstrings
- Created TROUBLESHOOTING.md for common issues (CORS, API keys, dependencies)
- Wrote INSTALL_AND_RUN.md for quick deployment
- Documented frontend features in LAYOUT_FEATURES.md
- Created CLAUDE.md for AI-assisted development guidance
- Added QUICK_COMMANDS.txt for common operations

Demo screenshots or a brief explanation of functionality

- Frontend Interface – Responsive 3-column layout with draggable splitters: map (left), places list (middle), chat (right)
- Chat Interface – User messages on right, AI responses on left, with streaming support showing "typing..." indicator during Gemini generation
- Interactive Map – Google Maps displaying all recommended places with numbered markers, route polylines connecting locations
- Places List – Collapsible cards showing place name, category, address, rating, hours, and Google Maps link
- Route Details – Expandable sections for each transport mode showing optimized waypoint order, total distance/duration, and leg-by-leg directions
- Transport Mode Switcher – 4 buttons (driving, walking, bicycling, transit) updating map polylines and route details instantly
- Session Management – Each browser session gets unique ID, maintaining separate chat histories and recommendations for concurrent users
- Streaming Responses – Real-time chat messages appearing token-by-token as Gemini generates response
- Google Search Grounding – Every place recommendation includes live data: exact name, address, rating, reviews, hours, Google Maps URL

Branching strategy

The team maintained the three-branch Git workflow established in Sprint 2:

- **main** → Stable production branch with tested releases
- **backend** → Active development branch for Sprint 3 work
- **feature/*** → Individual feature branches (gemini-migration, api-server, frontend-ui, route-optimization)

All pull requests were peer-reviewed and tested before merging into backend branch. The backend branch will be merged to main upon final Sprint 3 approval.

Challenges faced during development

- Gemini grounding prompts required multiple iterations to ensure Google Maps tool was invoked for every place (initial prompts allowed AI to skip grounding and use training data).
- Place ID extraction from grounding metadata required parsing multiple URL formats (cid parameter vs place_id parameter).
- Transit mode optimization required special two-step approach due to Google Maps API not supporting optimize_waypoints=True for TRANSIT mode.
- CORS errors during local development with file:// protocol required switching to http-server or Live Server extensions.
- Frontend map rendering occasionally lagged when displaying 10+ places simultaneously (solved with progressive marker loading).
- Google Maps API rate limits occasionally throttled requests during heavy testing (solved with API key monitoring and request throttling).
- Session storage in-memory means data loss on server restart (acceptable for Phase 1, will migrate to DynamoDB in Phase 2).
- Frontend CSS layout required significant iteration to achieve proper responsive behavior with draggable splitters.

Changes made to the original sprint plan

- Migrated from ChatGPT to Gemini AI mid-sprint for performance and grounding capabilities (original plan was to optimize ChatGPT integration).
- Implemented Place ID normalization system to handle Google Maps API returning different ID formats.
- Added background task for structured extraction instead of blocking /chat endpoint (improves user experience).
- Enhanced route optimization to support all 4 transport modes instead of just driving (original plan was driving + walking only).
- Created demo.html as standalone HTML file instead of React app (faster development, simpler deployment).
- Implemented in-memory session storage for Phase 1 instead of immediately jumping to DynamoDB (iterative deployment approach).
- Added comprehensive documentation suite (6+ markdown files) instead of just README.md.
- Implemented enforced Google Maps grounding with detailed system prompts to ensure Place ID availability.

GitHub & Trello links

GitHub Repository: https://github.com/Team5-TravelAssistant/Sprint3

Trello Board: https://trello.com/b/Sn0Gvt92/travel-assistant-tasks